

Documentacion de despliegue

Documentacion generada para administracion, usuarios y despliegue.

Identidad: dashboard financiero moderno, permisos por modulo, correo verificado, PostgreSQL y .NET 8.

Documentacion de despliegue - Finanzas Personales

Version: 1.1

Fecha: 2026-06-27

1. Objetivo

Esta guía explica como preparar, configurar y publicar la aplicacion Finanzas Personales en Railway.

Railway es una opcion adecuada para esta aplicacion porque permite desplegar servicios web con Docker y agregar PostgreSQL administrado en el mismo proyecto.

2. Arquitectura de despliegue recomendada

Arquitectura propuesta:

- Aplicacion web ASP.NET Core MVC en .NET 8.
- PostgreSQL administrado en Railway.
- Variables de entorno configuradas desde Railway.
- HTTPS y dominio publico gestionado por Railway.
- SMTP configurado desde la interfaz de la aplicacion.
- WhatsApp Business Cloud API configurado desde la interfaz de la aplicacion.

3. Consideracion importante sobre Railway y .NET

Railway soporta este despliegue usando Dockerfile.

Segun la guía oficial de Railway para ASP.NET Core, actualmente .NET debe desplegarse con Dockerfile porque Railpack todavia no soporta .NET. Tambien se debe escuchar en 0.0.0.0 y usar la variable PORT que Railway asigna al servicio.

Fuentes oficiales:

- <https://docs.railway.com/guides/asp-dotnet-core>
- <https://docs.railway.com/reference/errors/application-failed-to-respond>
- <https://docs.railway.com/guides/postgresql>

4. Archivos necesarios para Railway

El proyecto debe tener un Dockerfile en la raiz.

El Dockerfile debe:

- Compilar la solucion.
- Publicar el proyecto web.
- Copiar la documentacion y PDFs al contenedor.

- Ejecutar la aplicacion escuchando en 0.0.0.0:\${PORT}.

Railway detecta el Dockerfile y construye la imagen.

5. Variables de entorno necesarias

Ambiente

```
ASPNETCORE_ENVIRONMENT=Production
```

Cadena de conexion PostgreSQL

La aplicacion acepta cualquiera de estas dos opciones:

```
ConnectionStrings__Postgres  
DATABASE_URL
```

Opcion recomendada con formato Npgsql:

```
Host=HOST;Port=PORT;Database=DATABASE;Username=USER;Password=PASSWORD;SSL Mode=Require;Trust  
Server Certificate=true
```

En Railway puedes crear una base PostgreSQL y luego usar las variables generadas por Railway para formar la cadena.

Ejemplo conceptual:

```
ConnectionStrings__Postgres=Host=${{Postgres.PGHOST}};Port=${{Postgres.PGPORT}};Database=${{Po  
stgres.PGDATABASE}};Username=${{Postgres.PGUSER}};Password=${{Postgres.PGPASSWORD}};SSL  
Mode=Require;Trust Server Certificate=true
```

Opcion alternativa:

```
DATABASE_URL=${{Postgres.DATABASE_URL}}
```

La aplicacion convierte automaticamente postgres://... o postgresql://... al formato que usa Npgsql.

Administrador inicial

Recomendado en el primer despliegue:

```
InitialAdmin__Email=tu_correo@dominio.com  
InitialAdmin__Password=ClaveFuerteInicial123!
```

Despues de iniciar sesion por primera vez, cambia esa contrasena desde la aplicacion.

6. DataProtection y persistencia

La aplicacion usa ASP.NET Core DataProtection para cifrar:

- Contraseña SMTP.
- Token de WhatsApp.
- Datos protegidos internos.

Actualmente las llaves se guardan en:

```
App_Data/DataProtectionKeys
```

En Railway el filesystem del contenedor puede ser reemplazado en nuevos despliegues. Si se pierden esas llaves, puede ser necesario volver a configurar SMTP y WhatsApp.

Recomendacion para primera prueba:

- Aceptar esta limitacion durante pruebas.
- Si funciona bien, agregar volumen persistente o cambiar DataProtection a almacenamiento persistente externo.

Recomendacion para produccion:

- Configurar persistencia de llaves.
- No depender de llaves dentro de un contenedor efimero.
- Documentar procedimiento de rotacion o recuperacion.

Variable recomendada si montas un volumen en Railway:

```
DataProtection__KeysPath=/data/DataProtectionKeys
```

7. PostgreSQL en Railway

Pasos generales:

1. Crear proyecto en Railway.
2. Agregar servicio PostgreSQL.
3. Esperar a que Railway genere las variables de conexion.
4. Crear el servicio de la aplicacion web.
5. Configurar ConnectionStrings__Postgres o DATABASE_URL.
6. Desplegar.
7. Revisar logs de arranque.

La aplicacion ejecuta migraciones defensivas al iniciar:

- Crea columnas faltantes.
- Crea tablas auxiliares.
- Crea indices.
- Crea usuario administrador inicial si la tabla esta vacia.

Tambien existe script base:

```
sql/schema.sql
```

8. Paso a paso con GitHub y Railway

8.1 Preparar repositorio

1. Confirmar que el proyecto compila localmente.
2. Confirmar que el Dockerfile esta en la raiz.
3. Confirmar que appsettings.json no tiene contrasenas.
4. Subir el proyecto a GitHub.

Comandos locales:

```
cd "D:\DESARROLLOS PERSONALES\FinanzasPersonales"
dotnet build FinanzasPersonales.sln --no-restore
dotnet publish src\FinanzasPersonales.Web\FinanzasPersonales.Web.csproj -c Release -o
artifacts\publish
```

8.2 Crear proyecto en Railway

1. Entrar a <https://railway.com/>.
2. Crear nuevo proyecto.
3. Seleccionar despliegue desde GitHub.
4. Elegir el repositorio.
5. Railway detectara el Dockerfile.

8.3 Agregar PostgreSQL

1. Dentro del proyecto Railway, agregar PostgreSQL.
2. Revisar variables generadas.
3. Crear la variable `ConnectionStrings__Postgres` o `DATABASE_URL` en el servicio web.

Ejemplo:

```
ConnectionStrings__Postgres=Host=${{Postgres.PGHOST}};Port=${{Postgres.PGPORT}};Database=${{Postgres.PGDATABASE}};Username=${{Postgres.PGUSER}};Password=${{Postgres.PGPASSWORD}};SSL
Mode=Require;Trust Server Certificate=true
```

O:

```
DATABASE_URL=${{Postgres.DATABASE_URL}}
```

8.4 Configurar variables de la aplicacion

En el servicio web configurar:

```
ASPNETCORE_ENVIRONMENT=Production
InitialAdmin__Email=tu_correo@dominio.com
InitialAdmin__Password=ClaveFuerteInicial123!
ConnectionStrings__Postgres=...
```

Tambien puedes usar:

```
DATABASE_URL=...
```

No es necesario configurar PORT; Railway la asigna automaticamente.

8.5 Desplegar

1. Ejecutar deploy.
2. Revisar logs.
3. Confirmar que no aparecen errores de conexion a PostgreSQL.
4. Confirmar que la aplicacion responde.
5. Generar dominio publico desde Railway.

9. Dominio y HTTPS

Railway puede generar un dominio publico para pruebas.

Para produccion:

- Configurar dominio propio.
- Mantener HTTPS.
- Probar enlaces de verificacion de correo.
- Probar recuperacion de contrasena.
- Probar cookies de autenticacion.

10. SMTP

Configurar desde la aplicacion:

Administracion > Integraciones

Para Gmail:

- Servidor: smtp.gmail.com.
- Puerto: 587.
- SSL/TLS activo.
- Usuario: correo Gmail.
- Contraseña: contraseña de aplicacion, no contraseña normal.

Validar con el boton de prueba SMTP antes de crear usuarios reales.

11. WhatsApp

Configurar desde:

Administracion > Integraciones

Requiere:

- Cuenta Meta Business.
- WhatsApp Business Cloud API.
- Phone Number ID.
- Access Token.
- Plantilla aprobada.
- Telefono administrador.

Para mensajes automaticos fuera de la ventana de 24 horas se deben usar plantillas aprobadas por Meta.

12. Documentacion dentro de la aplicacion

La aplicacion muestra documentacion en:

Administracion > Documentacion

Para que funcione en Railway, el Dockerfile copia:

- docs
- output/pdf

Así se puede leer documentación en pantalla y descargar PDFs desde la aplicación.

13. Checklist antes de publicar en Railway

- Build local exitoso.
- Dockerfile en la raíz.
- Repositorio subido a GitHub.
- PostgreSQL creado en Railway.
- ConnectionStrings__Postgres o DATABASE_URL configurado.
- ASPNETCORE_ENVIRONMENT=Production.
- Administrador inicial configurado.
- SMTP probado.
- Recuperación de contraseña probada.
- Verificación de correo probada.
- Permisos probados con usuario no admin.
- HTTPS activo.
- Documentación visible en pantalla.
- PDFs descargables.
- Prueba móvil realizada.

14. Checklist posterior al despliegue

1. Abrir dominio Railway.
2. Iniciar sesión como administrador.
3. Cambiar contraseña inicial.
4. Configurar SMTP.
5. Enviar prueba SMTP.
6. Crear usuario de prueba.
7. Verificar que recibe correo de verificación.
8. Confirmar que no puede entrar antes de verificar.
9. Verificar correo.
10. Iniciar sesión con el usuario.
11. Confirmar que solo ve módulos permitidos.
12. Crear cuentas y categorías.
13. Registrar ingreso y gasto.

14. Crear recurrente de ingreso y gasto.
15. Revisar calendario.
16. Crear prestamo si tiene permiso.
17. Crear inversion si tiene permiso.
18. Abrir Documentacion en pantalla.
19. Descargar PDF funcional.

15. Problemas comunes en Railway

La aplicacion no responde

Revisar:

- Que escuche en 0.0.0.0.
- Que use \${PORT}.
- Que el Dockerfile ejecute la DLL correcta.
- Que no este usando solo localhost.

Error de base de datos

Revisar:

- Variable ConnectionStrings__Postgres o DATABASE_URL.
- Host, puerto, usuario, password y base.
- SSL requerido por la base.
- Logs de Railway.

SMTP no envia

Revisar:

- Contraseña de aplicacion de Gmail.
- Puerto 587.
- SSL/TLS activo.
- Salida de red permitida.

Se perdio configuracion SMTP o WhatsApp despues de redeploy

Probable causa:

- Llaves DataProtection no persistentes.

Solucion:

- Reconfigurar integraciones.
- Implementar persistencia de llaves para produccion.

16. Recomendacion final

Para la primera publicacion, Railway es una opcion adecuada para esta aplicacion si se despliega con Dockerfile y PostgreSQL administrado.

Para produccion real, antes de vender acceso a clientes, se debe resolver persistencia de DataProtection, backups de PostgreSQL, dominio propio, monitoreo y estrategia de recuperacion.